CGT 390 Lab 9 API and Data Design Report

Mahina Yasutake | Gatherly | October 22, 2026

Live prototype: cgt-390-project-seven.vercel.app

Backend preview: cgt-390-project-git-codex-lab9-backend-database-mahinay.vercel.app

Backend branch: github.com/MahinaYasutake/CGT390_Project/tree/codex/lab9-backend-database

Implementation status: The backend and database are deployed on the public live URL. Local, preview, and production checks passed on October 22. The peer test remains pending.

1. Interaction and contract

The task is to save Clay Club to a shortlist, see it after a reload, and remove it later. The revised prototype sends this action to a server API backed by an SQLite-compatible database.

Entry point

The Save button in app.js sends POST /api/saved with JSON {"eventId":"clay-club"}. api/saved.js validates the request, derives an anonymous visitor from an HttpOnly cookie, writes the database row, and returns the current IDs. GET /api/saved reads the list; DELETE /api/saved?eventId=clay-club removes one item.

Annotated flow

Save button → POST /api/saved → request validation and visitor cookie → events table ID check → saved_events insert → JSON response → Saved Events view. The public catalog supplies display details; saved_events stores the visitor-event relationship.

Contract

POST accepts exactly one field, eventId, as a string. A new save returns HTTP 201 with eventId, saved:true, and ids; a duplicate returns 200 with the same single saved row. DELETE returns 200 with saved:false and updated ids. A malformed body returns 400 INVALID_BODY; an unknown event returns 404 EVENT_NOT_FOUND; database unavailability returns 503 SAVE_SERVICE_UNAVAILABLE.

2. Data and access decisions

The events table contains catalog IDs. saved_events contains visitor_id, event_id, and created_at. Its (visitor_id, event_id) primary key prevents duplicates; event_id has a foreign key to events. The shared catalog.js array supplies public event details and seeds valid IDs in the database. The database is the source of truth for saved relationships.

The server creates a random visitor cookie and stores a hash of it as visitor_id. It never accepts a visitor ID from request JSON. Every read and write is scoped to that derived ID, so one anonymous browser cannot target another visitor's rows through the API. The cookie is HttpOnly and SameSite=Lax. This is anonymous ownership, not a signed-in account; clearing the cookie loses access to the list. No email, token, or payment field is returned.

3. Testing evidence

The API ran against a temporary real database on October 22. Predictions were compared with HTTP responses and follow-up reads. The sanitized local API log omits cookie values; the local browser log records screen behavior. The preview check and production check record hosted behavior. Core HTTP outcomes were repeated on production.

T1 Successful action

Input/identity: Visitor A POST {"eventId":"clay-club"}, then GET. **Prediction:** 201 and one saved ID after the read. **Actual:** 201 {"eventId":"clay-club","saved":true,"ids":["clay-club"]}; GET retained the ID. The local browser showed Saved 1 after reload. The insert and subsequent SELECT demonstrate persistence.

T2 Invalid input

Input/identity: Visitor A POST {"eventId":42}; then POST {"eventId":"not-real"}. **Prediction:** reject both and leave the list unchanged. **Actual:** 400 {"error":"INVALID_BODY"} and 404 {"error":"EVENT_NOT_FOUND"}; GET still showed ["clay-club"]. Type validation and the events-table lookup explain the two results.

T3 Access boundary

Input/identity: Visitor B GET, then DELETE clay-club; A GET again. **Prediction:** B starts empty and cannot remove A's save. **Actual:** B's initial GET returned {"ids":[]}; after B's delete, A's GET still returned {"ids":["clay-club"]}. A forged visitorId field in POST returned 400. The server-derived cookie identity and visitor-scoped SQL explain the boundary.

T4 Task-specific edge case

Input/identity: Visitor A repeats POST clay-club. **Prediction:** a second request leaves one row. **Actual:** HTTP 200 returned the same one-item list. The composite primary key and INSERT OR IGNORE explain the result. A direct insert with a nonexistent event ID was rejected by the foreign key.

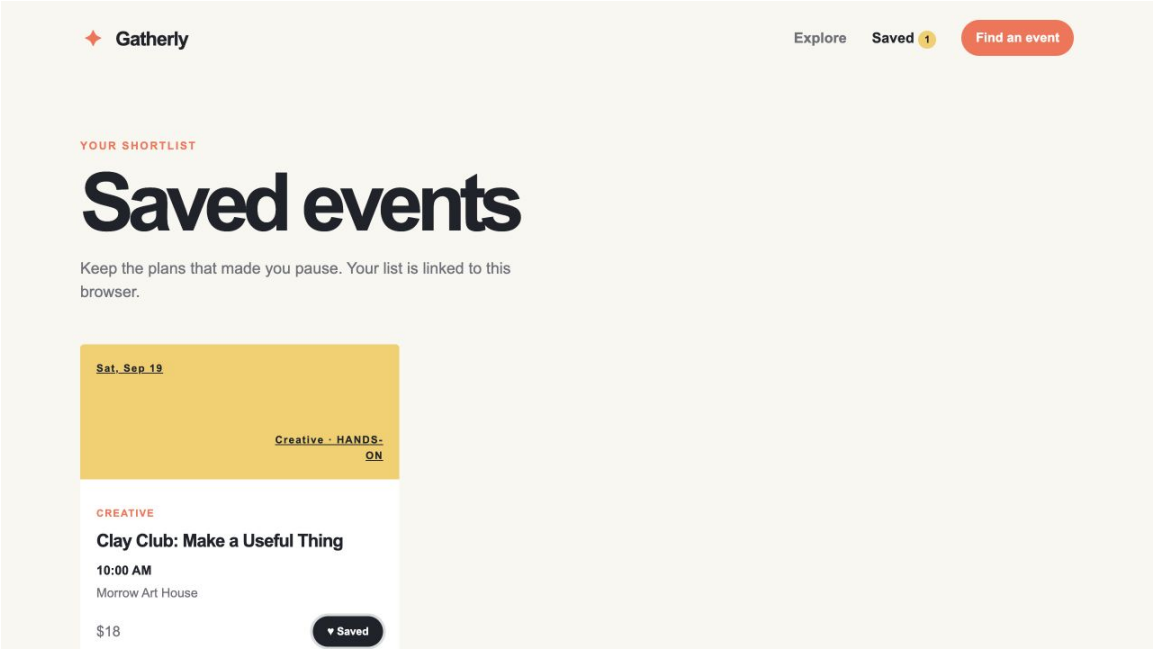
4. Findings and response

The original browser-storage review found two defects: invalid IDs were saved, and unknown detail links showed a different event. The before log shows ["not-real",""]; the retest log shows no write after the client fix. That repair is live.

Server validation and database constraints enforce the save rule. Four local browser checks passed: reload persistence, two-tab consistency, visitor isolation, and migration of old localStorage IDs. On the public URL, a save remained after reload, appeared on the Saved page, and disappeared after removal. Production API checks repeated success, duplicate, invalid, unknown, and access-boundary cases; the test row was removed.

Production browser evidence

The Saved page below shows Clay Club after the production save and page navigation. The browser then removed it and displayed Saved 0.



Remaining work: A classmate still needs to try one independent case. Anonymous cookies do not create accounts; reservations and capacity checks remain outside scope.

5. Peer review and AI use

Peer review

Pending: no classmate has yet proposed and run an additional case. The peer should try a fifth case and report the input, predicted and actual result, and any fix or clarification.

AI use

ChatGPT/Codex reviewed the code, built and ran the tests, implemented repairs, and drafted this report. The claims above were checked against the code, browser results, and stored values. I reviewed the document, decided on which action to investigate, and drafted parts of this report.